textOsFtextTOsFliningLFliningTLFtextosfliningIftabulartabproportionalprop
superiorSup
su-
pe-
ri-
or-
Sup

fontspechyperref

tool

Anonymous Author(s)*

ABSTRACT

1 INTRODUCTION

2 RELATED WORK

Large Language Models (LLMs) have demonstrated significant potential in code translation and generation tasks. Early studies highlighted LLMs' ability to act as "performance amplifiers" in code translation, showing that AI-supported engineers produced code with 50.8% fewer errors compared to manual translation, particularly in Java-to-Python tasks [**?**]. Subsequent research expanded this scope: LLMs like GPT-4 and CodeLlama outperformed traditional transpilers such as TransCoder in multi-language pairs, though success rates remained variable (2.1% to 47.3%) across benchmarks and real-world projects[**?**], [**?**]. Specialized applications emerged, including cross-language translation (e.g., Fortran to C++ [**?**], [**?**]) and repository-level conversion (e.g., C/Java/Python to Rust[**?**]). Tools like CodeScribe[**?**] and Oxidizer[**?**] leveraged LLMs to handle domain-specific challenges, such as preserving performance optimizations in scientific code and managing complex dependencies in large codebases. Evaluations on class-level translation [**?**] revealed that LLMs struggle with realistic, multi-component code, with computational accuracy dropping by 51.14–79.05% compared to method-level tasks, highlighting the need for task-specific optimizations that foreshadow the limitations explored in subsequent work.

Despite these advancements, LLMs face critical limitations in code-related tasks, primarily stemming from hallucinations, poor logical reasoning, and limited context length. Hallucinations in code generation—defined as outputs conflicting with intent, context, or knowledge—are prevalent across models, with studies showing LLMs like ChatGPT and CodeLlama struggle to detect or correct such errors effectively[**?**]. Logical reasoning deficits further impede performance. LLMs exhibit weaknesses in inductive reasoning[**?**] and often rely on context mimicry rather than genuine logical understanding, as demonstrated by their failure to maintain consistency when logical terms or contexts are modified[**?**]. In code translation, this manifests as errors in handling language-specific semantics, such as API mismatches, type conversions, and dependency invocations[**?**], [**?**]. Practical limitations include token constraints, which hinder translation of large codebases[**?**], and pre-training biases:for example, Python-oriented translations outperform C++/Java-oriented ones due to data imbalances[**?**]. All of above issues are exacerbated in long code translation (hundreds to thousands of tokens), especially in project-level translation, where even advanced models like GPT-4o achieve only 57.51% computational accuracy due to semantic loss[**?**]. This is because with the scale of code growing larger, Understanding and logical burden has sharply increased, at the same time, every small mistake can accumulate to bring unacceptable impacts to the entire project. Additionally, existing evaluation benchmarks (e.g., HumanEval) underestimate errors due to insufficient test coverage, masking the true extent of LLM limitations[**?**] and underscoring the urgency for effective enhancement strategies.

To address these limitations, researchers have developed diverse enhancement strategies, including retrieval-augmented generation (RAG), logical prompting, and iterative refinement. RAG-based approaches have shown particular promise: task-specific embedding alignment[**?**] improved CodeBLEU scores by 14–15% in Fortran-to-C++ translation, while triple knowledge augmentation (target-language samples, dependency examples, and successful translation pairs) boosted Pass@1 by 19.4% in repository translation[**?**]. Logical reasoning has been enhanced via prompting techniques like Logic-of-Thought (LoT)[**?**], which injects formal logic into contexts, improving performance on reasoning datasets by up to 8%. For code translation, UniTrans[**?**] integrated auto-generated test cases to mitigate logic comprehension issues, increasing computational accuracy by 4.02–28.58% across models. Similarly, PAST[**?**] used program state alignment via instrumentation to improve long code translation accuracy (e.g., GPT-4o from 57.51% to 84.70%). Preprocessing and post-processing techniques further refine outputs. Novel data preprocessing[**?**] and text augmentation[**?**] improved embedding quality and training data diversity, while iterative revision—such as the Translate-and-Revise framework[**?**]—reduced constraint violations by 15% in constrained translation tasks. Self-debugging mechanisms[**?**], [**?**] and hybrid approaches combining LLMs with rule-based validation[**?**] reduced errors by focusing on critical issues like type compatibility and dependency consistency.

Collectively, these studies highlight the dual nature of LLMs in code tasks: their transformative potential is tempered by inherent limitations, while emerging enhancements offer pathways to more reliable performance. However, gaps remain in addressing scalability for multi-file projects and building benchmarks for large-scale code translation, pointing to fertile areas for future research.

3 EMPIRICAL SETUP

In order to evaluate the effect of LLM code translation, we have to systematically translate a sufficient amount of code and use a unified standard to Statistically measure its success rate. We chose translating Java files into C++ files, both widely used programming languages which are extensively employed in many domains, such as system development and cross-platform application creation. We targeted two current mainstream LLMs (DeepSeek and ChatGPT) in code translation. The setup of our study includes three steps. The dataset collection step gathers datasets from open-source projects

on GitHub. The translation methods step uses two LLMs to translate code for these prompts. Finally, the evaluation metric step uses menu assessment to evaluate the translation results.

Our research is explored through two detailed aspects:

- **RQ1**: How well do existing LLMs perform in the code translation?
- **RQ2**: How to different prompt strategies affect the performance of LLMs in the code translation?

3.1 Target Projects

Both Java and C++ are widely used programming languages in diverse domains, such as:

- Development Tools and Frameworks
- Web Development Frameworks
- Data Science and Analysis

We believe that within mentioned domains, the selected projects represent key software components that are particularly well-suited for code translation. For each project, we applied the following detailed selection criteria to ensure fairness and objectivity:

- Minimal External Dependencies: Projects with few complex library dependencies are inherently easier to port.
- Cross-Domain Relevance: The functionality should be widely applicable across different languages and platforms, not tied to language-specific paradigms.
- Popularity and Recognition, split into three sub-criteria:
  a. Community Mentions and Usage: Frequency of discussions and references in forums, blogs, Q&A sites
  b. Dependency Downloads and Integration: Inclusion as a dependency in package managers
  c. Official Documentation and Tutorial Coverage: Presence in authoritative docs and tutorials

For example, `EnableJUnit4MigrationSupport` is the ideal choice as it's officially recommended by the JUnit team and widely adopted in enterprise environments. Recognized by top Java resources including Baeldung and DEV Community, it's the most reliable way to modernize your test suite without breaking existing tests. So we choose `EnableJUnit4MigrationSupport` as one of the Java datasets.

Besides, in terms of project scale, to avoid difficulties in subsequent processing caused by overly large scales and to prevent the lack of typicality due to overly small scales, the selected datasets includes several projects of moderate scale. The specific scales of the project are in Table **??**.

Thus, a high-quality and highly representative Java project datasets is obtained for subsequent research.

3.2 Evaluated Models

We selected two state-of-the-art large language models (LLMs) for evaluation: GPT-4 and DeepSeek-Coder-V3. GPT-4 is a proprietary, general-purpose model developed by OpenAI, known for its strong reasoning and few-shot capabilities across a wide range of natural language and code-related tasks. While OpenAI has not disclosed GPT-4's parameter size, it is considered significantly larger than prior models like GPT-3.5 and trained on a mixture of natural language and programming language data. DeepSeek-Coder-V3, on the other hand, is an open-source, code-specialized LLM developed by DeepSeek, optimized for programming understanding and generation. DeepSeek-Coder-V3 is available in multiple sizes; for this study, we selected the 7B Instruct variant, which is fine-tuned for instruction-following tasks in code translation and completion scenarios. The selected models are summarized in Table 2, including their parameter sizes, training data characteristics, and open-source availability.

Table 1: Project Dataset Statistics

Domains	Projects	Number of files
Development tools and frameworks	EnableJUnit4MigrationSupport	32
	NoopIndexDAO	21
Web development frameworks	cookie	10
	HoverMenuService	9
Data science and analysis	IntMath	7

3.3 RQ1 Setup

To evaluate the translation quality of Large Language Models (LLMs) in realistic settings, we design a task that mirrors end-to-end class-level code translation. Specifically, each Java class file is treated as an atomic translation unit, containing both interface declarations and full method bodies. For each selected Java file, we prompt the LLMs with the complete content, instructing them to generate corresponding C++ header and source files. The format explicitly requires:

- Structured separation into *.h and *.cpp components
- Omission of any non-code explanation or usage examples
- Conformity with C++17 syntax conventions

This setup ensures that the model must understand the Java class holistically, including its internal logic, data types, visibility modifiers, and any implicit dependencies. This experimental setting tests the LLMs' ability to handle coherent, multi-method class translation, which is considerably more challenging than isolated method-level conversion. Prior studies have shown that LLMs perform worse on class-level tasks due to increased complexity and longer token contexts (Liu et al. 2023). For example, evaluations on class translation have reported a 51.14–79.05% drop in computational correctness compared to method-level tasks due to semantic inconsistency and hallucinations (Chen et al. 2023; Wu et al. 2023). Moreover, large language models often struggle with preserving internal structure, managing variable scope across methods, and maintaining cross-reference integrity—issues that are not prominent in method-based tests (Ding et al.

2023). As such, class-based translation serves as a strong indicator of model robustness and practical deployability in codebase refactoring tasks. To further ensure fairness, we fix the decoding strategy (greedy decoding, temperature = 0) and maintain identical prompts across both ChatGPT and DeepSeek-Coder. The evaluation relies on post-translation manual review guided by a structured error taxonomy introduced in Section 4. This setup directly addresses RQ1: How well do existing LLMs perform in full class-level code translation from Java to C++? It enables us to assess not only translation fidelity but also structural completeness and hallucination tendencies. Unlike simplistic method-level tests, this approach highlights nuanced issues such as incorrect memory management, misaligned access specifiers, and syntactic loss across inheritance hierarchies—providing a realistic benchmark of LLM capabilities in practical software engineering scenarios.

3.4 RQ2 Setup

To address these research questions, we conducted a series of translation involving two LLMs (DeepSeek and ChatGPT). For each LLMs, we translate in two ways:

3.4.1 *Class-Based Translation.*

Prompt of Class-Based Translation. The raw data files were generated by scripts that traverse specified project directories, treating each Java file as an independent translation unit.

- Instruction: "Translate the following Java code into C++:"
- Source Code: The entire content of the target Java file.
- Format Requirements: "Your answer should include two parts: header file and source file (if the given java file contains an interface, source file are not required), every part should be surrounded by cpp. assuming that the relevant dependencies have been implemented, provide the code only, no other non code content is required, and no usage examples are required."
- Example: Finally, an "example" string is attached, providing a sample of the expected C++ header and source file

Header File Generation.

- LLMs parse Java files
- Translate elements (class definitions, member variables, method declarations) into C++ header files
- Follow C++ conventions

Class-Based Translation.

- Using the generated header files
- LLMs receive the entire Java class as input
- Translate the complete class structure, including fields and methods, into C++

Evaluation and Error Analysis.

- Evaluating translation consistency and completeness
- Analyzing errors
- Comparing the effectiveness, robustness, and granularity of both strategies

3.4.2 *Method-Based Translation.*

Prompt of Method-Based Translation. The raw data files were generated by scripts that traverse specified project directories, extracting individual Java classes or interfaces and their method bodies. Each Java file is separated into two parts: a header file section and a body section.

- Instruction: To the header file section: "Following is a Java class/interface declaration along with field and method declarations:{code} Please translate it into a header file that conforms to the C++17 standard syntax and performs the same functions. The method only needs to be declared, not implemented. Other classes that appear in the code have been implemented in C++. If necessary, you can use the header file with the same name."
  To the body section: "{header_content}Please implement the following functions according to the contents of the above header file:codeIt is a Java method that you need to translate into a member function that conforms to the C++17 standard syntax. Other classes that appear in the code have been implemented in C++. If necessary, you can use the header file with the same name. Only the in vitro implementation of the function is required, no other non-code content is required, and no usage examples are required."
- Source Code: The entire content of the target Java file.
- Format Requirements: "Your answer should include two parts: header file and source file (if the given java file contains an interface, source file are not required), every part should be surrounded by cpp. assuming that the relevant dependencies have been implemented, provide the code only, no other non code content is required, and no usage examples are required."
- Example: Finally, an "example" string is attached, providing a sample of the expected C++ header and method implementation.

Header File Generation.

- LLMs parse Java files
- Translate elements (class definitions, member variables, method declarations) into C++ header files
- Follow C++ conventions

Method-Based Translation.

- Using the generated header files
- LLMs translate individual Java methods independently
- Each method is translated into C++ without full class context

Evaluation and Error Analysis.

- Evaluating translation consistency and completeness
- Analyzing errors
- Comparing the effectiveness, robustness, and granularity of both strategies

Both approach ensures a systematic and structured conversion from Java to C++.

3.5 Evaluation Metric

The evaluation of the translation results primarily relies on manual assessment, supported by a structured menu assessment approach to ensure consistency and reduce evaluator bias.

The evaluation process includes:

- Line-by-line scrutiny comparing C++ code against original Java files
- Systematic categorization of error types
- Statistical analysis of identified issues

To standardize the process, a menu assessment framework is introduced. This framework provides evaluators with:

- A checklist of common translation criteria such as Reference to non-existent methods or member variables, Parameter table does not match, Missing header files and so on.
- Examples of acceptable and unacceptable translations
- For parts that are difficult to understand or easily confusing, mark notes (e.g., Note: This type of error should be distinguished from types 1,9, and 10: if the problem is solved after adding the missing header file, it is counted as type 3. If there is no such header file, it means that the function or class used is fabricated out of thin air, which is counted as 1, 9, and 10. (Note: These two cases still do not include classes or methods that exist in the project))

This evaluation model:

- Enhances the reliability and repeatability of manual assessments
- Effectively reflects performance disparities among different LLMs
- Offers clear, actionable insights to guide improvements in code translation technologies

This evaluation framework supports in-depth analysis of (RQ1) and (RQ2). Nevertheless, we recognize the value of exploring this issue and leave a broader question for future work: how can we reduce reliance on manual evaluation while still ensuring accurate and reliable assessment of code translation quality?

4 DATA ANALYSIS

4.1 Introduction/Overview

This section aims to quantitatively analyze and compare the performance of large language models (LLMs) in Java to C++ code translation tasks. By manually analyzing and annotating the types of hallucinations generated during the translation process, compiling statistics, and calculating the frequency of various errors, we further analyze the key factors influencing LLM performance. This provides empirical evidence for evaluating the current capabilities and limitations of LLMs in code project translation.

The dataset analyzed in this study is derived from the work of researchers who translated methods from 5-6 Java projects into C++ using different LLMs and methods. This dataset comprises a total of 1994 translated methods, which have undergone meticulous manual error type annotation.

We systematically explore LLM performance across three primary analytical dimensions to comprehensively understand translation efficiency and hallucination patterns:

- **Model:** Compares the performance of GPT (representing widely adopted general-purpose LLMs) and DeepSeek (an emerging specialized or alternative LLM).
- **Error Type:** Employs a detailed, manually identified error classification scheme to categorize and annotate the most common errors occurring during the translation process by existing large language models. This aids in precisely understanding the common errors and concentration of LLM hallucinations.

4.2 Statistical Methods

The quantitative analysis in this study involves calculating metrics for evaluating LLM translation hallucinations: correctness rate and error rate, which are defined as the number of correctly and incorrectly translated methods divided by the total number of methods, respectively. These metrics were calculated for different groupings to facilitate further comparative analysis.

4.3 Identified Errors

To systematically quantify and analyze the "hallucination" phenomenon of large language models in Java to C++ code translation tasks, we established a detailed error classification standard. Through meticulous manual review of the translation results, we identified and summarized several core error types that repeatedly occurred during the translation process.

4.3.1 Referencing non-existent methods or member variables.

This error type refers to calling a method in the translated code that does not exist in a class or its base class. This usually happens when the model attempts to directly migrate a specific API or programming habit from the source language to the target language, but the corresponding class library in the target language does not have a functionally equivalent member function.

```
1  class MyClass {
2  public:
3      void doSomething() {
4          // ...
5      }
6  };
7
8  void example_type1() {
9      MyClass obj;
10     obj.doSomethingElse(); // Compilation error: 'class MyClass' has no member named
11 'doSomethingElse'
12 }
```

4.3.2 Parameter list mismatch. When code calls a function or method, the provided parameters do not match the parameter list of any available overloaded version of that function in terms of quantity, type, or order.

```
void processData(int a, double b) {
    // ...
}

void example_type2() {
    processData(10, "hello"); // Compilation error: cannot convert 'const char*' to 'double' for argument 2
    processData(5);          // Compilation error: too few arguments to function call
}
```

4.3.3 Missing header file. The code uses a class, function, or macro defined in the C++ standard library or other parts of the project, but does not include the corresponding header file via an #include directive, causing the compiler to fail to find the definition of related symbols, thus leading to compilation failure. This error needs to be distinguished from "Referencing non-existent classes": in type 3, the referenced symbol itself exists, only the declaration is missing; while in type 7, the referenced class is entirely fictitious.

```
// Assume the <vector> header file is forgotten.
void example_type3() {
    std::vector<int> myVec; // Compilation error: 'vector' is not a member of 'std' (if <vector> is not included)
}
```

4.3.4 Method not implemented (entirely missing). This error type refers to a member function being declared in the class's header file or source file, but no implementation for that function is provided at all in the corresponding source file, or the implementation body is empty.

```
class AnotherClass {
public:
    void initialize(); // Declared, but no implementation in the .cpp file
    void cleanup();
};

// In the corresponding .cpp file, if the following implementation is missing:
void AnotherClass::initialize() { }
// Then a linking error will occur: undefined reference to `AnotherClass::initialize()`
```

4.3.5 Variable initialization/usage error. This error type typically includes not initializing reference members or constant members in the constructor's member initializer list. Additionally, using uninitialized variables also falls into this category.

```
class Config {
public:
    const int value; // Const members must be initialized in the constructor's member initializer list
    int& ref;       // Reference members must be initialized in the constructor's member initializer list

    Config() {} // Compilation error: 'value' and 'ref' must be initialized in constructor base/member initializer list
    }
};

void example_type6() {
    int data = 10;
    Config c; // If there is no constructor with parameters above, and the default constructor does not initialize const/reference, it will cause an error.

    int uninitialized_var;
    int result = uninitialized_var + 5; // Runtime error: using uninitialized variable, undefined behavior
}
```

4.3.6 Incorrect pointer usage. This error type involves improper operations on C++ pointers (including raw pointers and smart pointers). For example, dereferencing a void pointer and attempting to call its member method, or incorrectly performing pointer type conversions.

```
class Base {
public:
    virtual void print() { /* ... */ }
};

class Derived : public Base {
public:
    void print() override { /* ... */ }
    void specificMethod() { /* ... */ }
};

void example_type8() {
    void* rawPtr = new Derived();
    static_cast<Derived*>(rawPtr)->specificMethod(); // Compiles, but runtime error if rawPtr does not actually point to a Derived object
    rawPtr->print(); // Compilation error: 'void*' is not a pointer-to-object type, cannot dereference

    Base* basePtr = new Base();
    Derived* derivedPtr = static_cast<Derived*>(basePtr); // Compiles, but runtime error if basePtr does not actually point to a Derived object
    derivedPtr->specificMethod(); // Runtime error: attempting to call a non-existent method (if the object is not Derived)

    delete rawPtr; // Free memory
    delete basePtr;
}
```

4.3.7 Referencing non-existent classes. This error type refers to the code referencing a class that is not defined in the C++ standard library, project dependencies, or the current codebase. This is usually because the translation directly copied Java-specific libraries, and these classes do not have direct corresponding implementations in C++, requiring the use of different libraries or design patterns as alternatives.

```
void example_type9() {
    NonExistentClass obj; // Compilation error: 'NonExistentClass' was not declared in this scope
}
```

4.3.8 Method partially unimplemented (partially missing). Unlike "Method not implemented (entirely missing)", this error type refers to methods that have an implementation body, but their internal logic is incomplete, contains placeholders, or is severely simplified, failing to achieve the full functionality of the original Java code.

```
class DataProcessor {
public:
    void processData(int input) {
        // TODO: Add actual processing logic here
        // This is a placeholder, actual logic is missing.
        int result = input * 2;
    }
};
```

4.3.9 Improper memory management. This error type specifically targets C++'s manual memory management mechanism. It primarily includes memory leaks, double-freeing the same memory block, or using dangling pointers that have been freed.

```
void example_type11() {
    // Memory leak
    int* leakyPtr = new int[10];
    Forgot to delete[] leakyPtr; // Leads to memory leak

    // Double free
    int* doubleFreePtr = new int;
    delete doubleFreePtr;
    delete doubleFreePtr; // Runtime error: double free or corruption

    // Using dangling pointer
    int* danglingPtr = new int;
    delete danglingPtr;
    *danglingPtr = 10; // Runtime error: using freed memory (dangling pointer)
}
```

4.4 Overall Performance

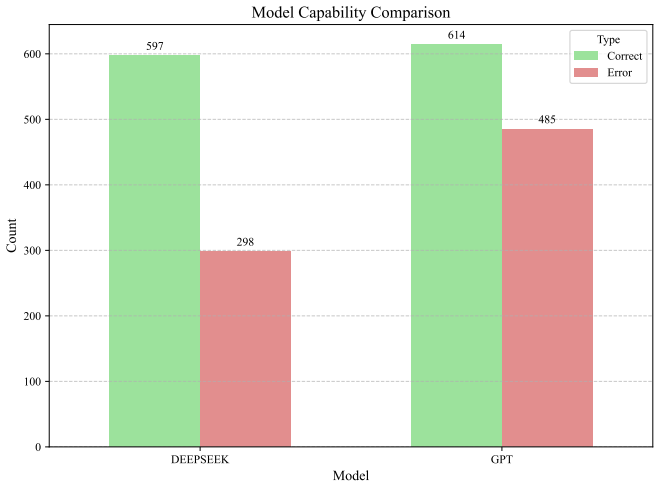
4.4.1 Overall Correctness Rate/Error Rate. Across all evaluated LLMs (GPT and DeepSeek), a total of 1994 distinct methods were translated and analyzed. Of these, 1211 methods were accurately translated, while 783 methods contained one or more identified errors. The overall correctness rate was 60.73%, and the error rate was 39.27%. This data establishes a baseline reference for LLM performance in Java to C++ code translation.

Metric	Quantity	Proportion (%)
Correctly Translated Methods	1211	60.73
Incorrectly Translated Methods	783	39.27

Based on data analysis, a significant portion of methods were accurately translated, though a notable percentage contained identified errors. These findings are important for understanding current LLM proficiency in cross-language code conversion and provide reference points for future advancements.

4.4.2 Performance w.r.t LLMs. Based on existing translation results, we can compare the overall translation efficiency of GPT and DeepSeek.

- **GPT:** GPT processed a total of 1099 methods. Of these, 614 methods were correctly translated, and 485 methods were incorrectly translated. This gives GPT an overall correctness rate of 55.87%.
- **DeepSeek:** DeepSeek processed a total of 895 methods. It completed 597 correct translations and 298 incorrect translations. This gives DeepSeek a higher overall correctness rate of 66.70%.



DeepSeek's overall correctness rate (66.70%) was significantly higher than GPT's (55.87%), demonstrating an absolute advantage of 10.83 percentage points. This indicates that, in this experimental setup, for the methods it completed, DeepSeek was, on average, more reliable in generating correct Java to C++ translations.

4.4.3 Performance w.r.t Projects. To investigate how project characteristics affect translation performance, we conducted a comparative analysis of the translation results across five different projects. The table below summarizes the correctness rate and error distribution for each project under the class strategy.

Based on the data, the EnableJUnit4MigrationSupport project, which has the highest file count (32), also exhibits nearly the lowest correctness rate (46.7%). This preliminarily suggests that as project scale increases, contextual dependencies may become more complex, leading LLMs to make more errors during holistic class-level translation, particularly generating a large number of "Method not implemented (entirely missing)" and "Referencing non-existent classes" errors. Similarly, the second-largest project, NoopIndexDAO, while having a decent correctness rate (69.8%), also suffers from a high volume of "Method not implemented (entirely missing)" errors.

Although the IntMath project has the fewest files (7), its translation correctness rate is also very low (47.8%), with errors almost entirely concentrated in "Method not implemented (entirely missing)" (accounting for 93.8% of its total errors). This may imply that the project involves numerous Java-specific design patterns and libraries, as well as fundamental differences from C++ in memory management, null handling, and standard library functionalities. When faced with methods requiring cross-language paradigm shifts and idiomatic C++ implementations, LLMs struggle to find direct correspondences, thus tending to omit their implementation.

The Cookie project stands out with the highest correctness rate of 86.5%. As a smaller-scale project (9 files), its error types are also more dispersed, without a single error type being overwhelmingly dominant. This might indicate that

Table 2: Error Types

Error Type	Entity	Description
Referencing non-existent methods or member variables	Method Call / Member Access	Calling a method or accessing a member variable that does not exist in a class or its base class.
Parameter list mismatch	Method Declaration	The parameters provided in a function call do not match the quantity, type, or order of any available overloaded version of that function.
Missing header file	#include Directive	The code uses a class, function, or macro but fails to include the corresponding header file, leading to a compilation failure.
Method not implemented (entirely missing)	Method Definition	A member function is declared, but no implementation is provided for it, or the implementation body is empty.
Variable initialization/usage error	Variable Declaration / Constructor	Failure to initialize reference or constant members in a constructor's initializer list, or using any variable before it has been initialized.
Incorrect pointer usage	Pointer Operation	Improper operations on C++ pointers, such as dereferencing a void pointer or performing incorrect type conversions.
Referencing non-existent classes	Class Reference / Type Usage	The code references a class that is not defined in the standard library, project dependencies, or the current codebase.
Method partially unimplemented (partially missing)	Method Implementation	The method has an implementation body, but its internal logic is incomplete, contains placeholders, or is simplified, failing to match the original functionality.
Improper memory management	Memory Allocation/Deallocation	Errors related to manual memory management, including memory leaks, double-freeing memory, or using dangling pointers.

the project's code structure is relatively simple with clear dependencies, making it more suitable for LLM translation.

In summary, under the class strategy, project scale and content complexity are key factors influencing translation quality. For large-scale or logically complex projects, LLMs struggle to maintain a complete contextual understanding, leading them to frequently omit method implementations, which severely impacts the completeness and usability of the code.

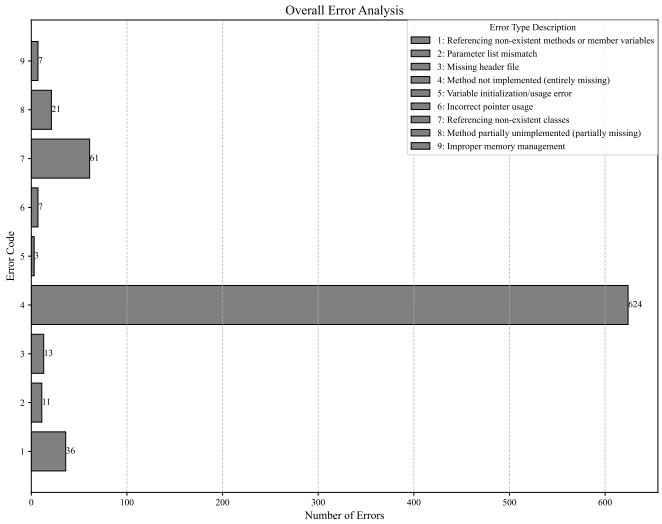
4.4.4 Error Type Distribution. Analyzing and comparing the distribution of all identified error types across the entire translation corpus, we observed that certain hallucination categories predominated. "Method not implemented (entirely missing)" was the most prevalent, accounting for a total of 624 instances. This single error type constituted approximately 51.53% of all identified errors, revealing the current LLMs' inadequacy in generating complete code structures.

Other significant error types include:

- **Referencing non-existent classes:** This category accounted for 61 instances. This type primarily arises from LLMs using non-existent classes in C++ when generating translated files. These may stem from differences between Java and C++ or be entirely a product of LLM hallucination. This error indicates that LLMs still struggle with correctly identifying or hallucinating external class dependencies.
- **Referencing non-existent methods or member variables:** 36 instances. This error indicates difficulty in maintaining member access consistency within classes.
- **Method partially unimplemented (partially missing):** This error type occurred 21 times, indicating cases where a method was generated but not completed.

Other less common but still noteworthy error types included "Missing header file" (13 instances) and "Parameter list mismatch" (11 instances).

Project Name	File Count	Correct Methods	Erroneous Methods	Correctness Rate	Dominant Error Types (Count)
Cookie	9	166	26	86.5%	Missing header file(6), Improper memory management(4)
EnableJUnit4MigrationSupport	32	293	334	46.7%	Method not implemented (entirely missing)(224), Referencing non-existent classes(55)
HoverMenuService	9	58	50	53.7%	Method not implemented (entirely missing)(44)
IntMath	7	89	97	47.8%	Method not implemented (entirely missing)(91)
NoopIndexDAO	21	609	264	69.8%	Method not implemented (entirely missing)(253)



The overwhelming dominance of "Method not implemented (entirely missing)" is not just a quantitative finding but also qualitatively highlights fundamental limitations of LLMs. This suggests that when faced with complex class translation tasks, LLMs often opt to omit entire methods rather than attempting to translate them. This also points to a structural hallucination, where the model completely fails to produce the expected output, rather than merely generating semantically or syntactically incorrect code.

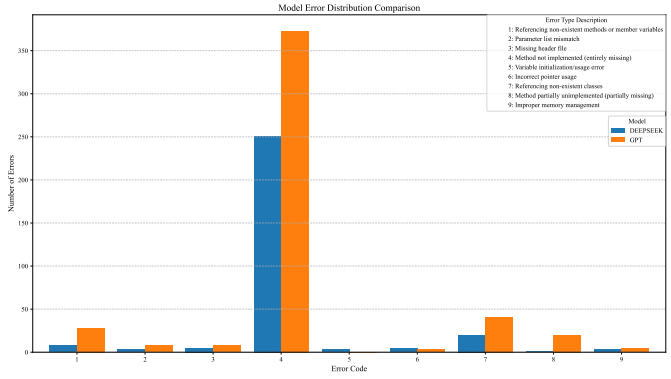
If LLMs frequently omit entire methods, it implies significant challenges in maintaining long-term coherence and completeness in medium to large-scale code projects. The occurrence of this error may stem from several factors: First, the LLM's context window might be insufficient to accommodate the entire class definition and its dependencies, preventing a complete analysis of methods and leading to the omission of their entire implementation. Second, when target methods are not explicitly referenced or prompts are too broad, LLMs may fail to fully grasp the implicit requirements of all methods within the translated class. Finally, in situations of high uncertainty or perceived complexity, LLMs struggle to fully convert Java language features to C++. They might choose to omit or fill with TODOs for later completion rather than generating highly erroneous or uncompilable code. This type of hallucination is particularly critical as it results in incomplete or non-functional target code, requiring substantial manual intervention.

The high incidence of method omission suggests that, without explicit, fine-grained guidance, current LLMs may lack an internal coherent understanding of software project structure and code completeness, making them less adept at automating large, multi-component code migration tasks. While they excel at generating syntactically plausible code snippets, their ability to generate complete, coherent, and functionally equivalent project systems remains nascent. To complete the translation and migration of an entire project, LLMs need to understand the complete structure of the target code, not just the internal logic of individual methods. Future LLM development needs to focus on improving their ability to understand code as a structured system, rather than merely translating raw code as a series of independent fragments.

4.4.5 Error Type Distribution by Model. A detailed examination of the error type distribution for each model reveals different characteristics in their hallucination frequencies.

- **GPT Error Distribution:** GPT's most common error was "Method not implemented (entirely missing)," with 373 instances. Other significant errors included "Referencing non-existent classes" (41 instances) and "Referencing non-existent methods or member variables" (28 instances).

- **DeepSeek Error Distribution:** For DeepSeek, "Method not implemented (entirely missing)" was also the most common error, but its count was significantly lower than GPT's, at 251 instances. Other errors included "Referencing non-existent classes" (20 instances) and "Referencing non-existent methods or member variables" (8 instances).



DeepSeek demonstrated a stronger ability to prevent complete method omissions, with 122 fewer "Method not implemented (entirely missing)" errors than GPT. This factor contributed to DeepSeek's higher overall correctness rate.DeepSeek in code translation accuracy and completeness showed a clear advantage, especially in avoiding "Method not implemented" and "Referencing non-existent" errors. GPT in this type of basic translation method appears to have a higher error rate, especially in generating complete method implementations. If the main goal is to efficiently obtain compilable and functionally correct translated code, then DeepSeek seems to be the preferred choice.

4.5 Data Analysis Summary

- **Pervasive Hallucinations:** Large Language Models (LLMs), including GPT and DeepSeek, exhibit a significant hallucination rate in Java to C++ code translation. The most prevalent and impactful error type is "Method not implemented (entirely missing)." This phenomenon indicates a deficiency in LLMs' ability to fully comprehend code project structures and migrate complete code.
- **Model-Specific Strengths:** DeepSeek generally outperformed GPT in overall correctness, primarily due to its lower rate of "Method not implemented" errors, revealing DeepSeek's stronger inherent ability to maintain code completeness.

4.6 Analysis Based on Method Strategy

To further investigate the impact of translation granularity on the performance of large language models, we introduced a translation strategy based on individual methods and conducted a comprehensive data analysis of its results. This strategy involves extracting each Java method independently and instructing the model to translate it into the corresponding C++ method. This chapter will detail the overall performance under this strategy, compare the performance of different models, and conduct a horizontal comparison with the previously discussed class-based translation strategy to reveal how different strategies affect model translation behavior and hallucination patterns.

4.7 Overall Performance

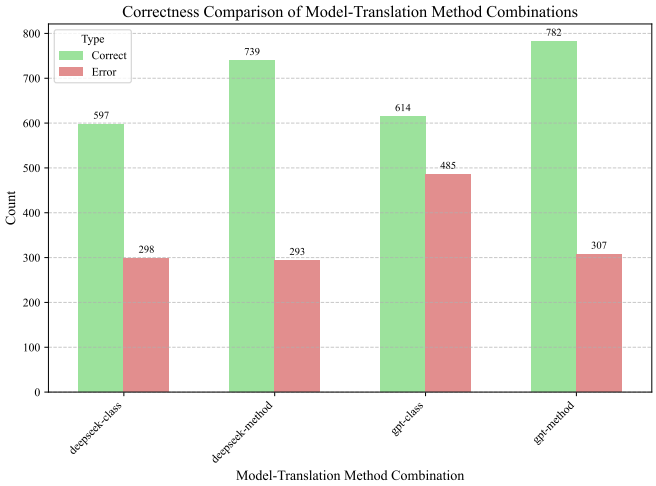
4.7.1 Overall Correctness Rate/Error Rate. When translating the dataset using the method strategy, a total of 2121 methods were processed. Among them, 1521 methods were translated correctly, while 600 contained errors, resulting in an overall correctness rate of 71.71% and an error rate of 28.29%.

Metric	Quantity	Proportion (%)
Correctly Translated Methods	1521	71.71
Incorrectly Translated Methods	600	28.29

The overall correctness rate of the method strategy (71.71%) is significantly higher than that of the class strategy (60.73%), an absolute improvement of about 11 percentage points. This result indicates that breaking down the translation task into smaller, more focused units (individual methods) can effectively increase the probability of LLMs generating correct code. When the model only needs to focus on the internal logic of a single method, the complexity is significantly reduced, thereby lowering the likelihood of errors. This demonstrates that refining the translation granularity is an effective way to enhance LLM performance in code migration tasks.

4.7.2 Performance w.r.t LLMs. Under the method translation strategy, we also compared the performance of the GPT and DeepSeek models:

- **GPT:** GPT processed a total of 1089 methods, with 782 correct and 307 incorrect, for a correctness rate of 71.81%.
- **DeepSeek:** DeepSeek processed a total of 1032 methods, with 739 correct and 293 incorrect, for a correctness rate of 71.61%.



This result presents a stark contrast to the model performance under the class strategy. In the class strategy, DeepSeek's correctness rate (66.70%) was significantly superior to GPT's (55.87%). However, under the method strategy, the two models' correctness rates are surprisingly close, with

almost no significant difference; GPT's performance is even slightly better.

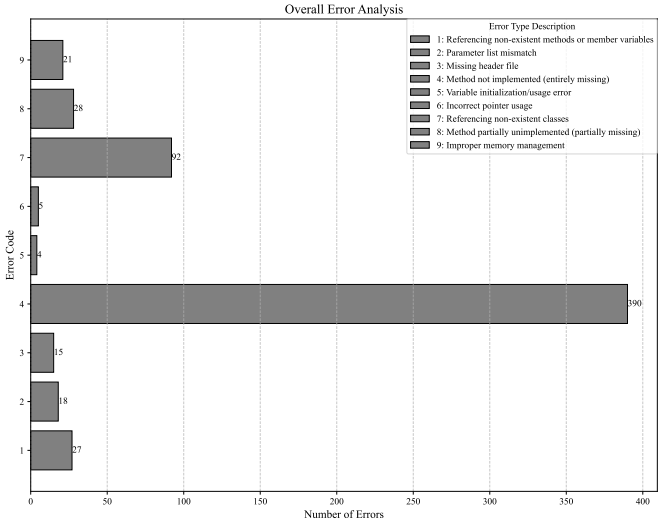
This reversal of trends indicates that translation granularity is a key variable affecting the relative performance of the models. DeepSeek shows an advantage when handling more complex, context-rich class-level translation tasks, which may be due to its stronger ability to understand the overall code structure. However, when the task is simplified to a method-level translation with limited contextual information, GPT demonstrates equivalent or even slightly stronger competitiveness. This suggests that GPT is highly efficient at handling local, independent logic fragments, while DeepSeek's advantages are not fully realized when information is constrained.

4.7.3 Performance w.r.t Projects. Under the method strategy, we also analyzed the translation performance across the five projects to assess how the change in translation granularity affects different types of projects.

Project Name	File Count	Correct Methods
Cookie	9	137
EnableJUnit4MigrationSupport	32	333
HoverMenuService	9	195
IntMath	7	135
NoopIndexDAO	21	724

The correctness rate for all projects improved under the method strategy. The most significant improvements were seen in HoverMenuService (jumping from 53.7% to 88.6%) and IntMath (from 47.8% to 67.8%). This strongly proves that decomposing the task into smaller units (methods) effectively reduces translation complexity, with more pronounced effects on projects that performed poorly under the class strategy. The change in translation granularity also led to a shift in error patterns. For instance, the primary error in the HoverMenuService project shifted from "Method not implemented (entirely missing)" under the class strategy to "Referencing non-existent methods or member variables" under the method strategy. This highlights a critical issue: when the context is limited to a single method, the model is less likely to omit the implementation, but its lack of awareness of the entire class makes it more prone to hallucinate when handling inter-method call relationships. Despite the improved correctness rate, the EnableJUnit4MigrationSupport project (56.7%) still performs the worst. Its errors remain concentrated in "Method not implemented (entirely missing)" and "Referencing non-existent classes," indicating that even when the task is decomposed, LLMs still face significant challenges in inferring missing context and dependencies for inherently complex projects. The correctness rate of the Cookie project decreased under the method strategy, which contradicts the general trend of improvement in other projects. Its primary error is "Improper memory management," which may suggest that the project contains many low-level operations requiring careful memory handling. Under the class strategy, the full context helps the model understand these operations, whereas, under the method strategy, the lack of information leads to more of these subtle logical errors. In conclusion, the method strategy generally improves the translation correctness rate across projects by reducing task complexity. However, this strategy also amplifies the model's hallucination problem when dealing with local dependencies and may negatively affect specific code (such as memory management) that relies on a complete context.

4.7.4 Error Type Distribution. Among the 600 errors produced by the method strategy, "Method not implemented (entirely missing)" remains the most numerous error type, with 390 instances, accounting for 65% of all errors. This is followed by "Referencing non-existent classes," with 92 instances, making up 15.33%. Other error types such as "Method partially unimplemented (partially missing)" (28 instances) and "Referencing non-existent methods or member variables" (27 instances) also account for a certain proportion, but their numbers are small compared to the first two categories.

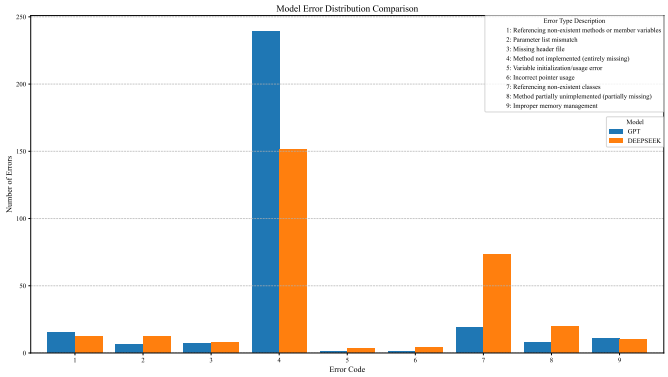


Similar to the error distribution of the class strategy, "Method not implemented (entirely missing)" is also the primary form of hallucination in the method strategy. However, it is worth noting that the proportion of this error in the method strategy (65%) is much higher than its proportion in the class strategy (approximately 51.53%). This reveals a key phenomenon: when contextual information is limited to a single method, if the model cannot resolve the method's external dependencies (such as other classes, global variables, etc.), it is more inclined to abandon code generation altogether rather than attempting a potentially incorrect implementation. This "safe" failure strategy, while leading to a large number of missing methods, may also be one of the reasons for its higher overall correctness rate, as it avoids generating more functionally incorrect code containing other types of errors.

Furthermore, the relatively high proportion of "Referencing non-existent classes" errors (15.33%) highlights the significant challenges models face in inferring and reconstructing external dependencies when a complete class definition context is lacking.

4.7.5 Error Type Distribution by Model. Although the two models have similar overall correctness rates under the method strategy, their error patterns reveal distinct weaknesses.

- **GPT Error Distribution:** Among its 307 errors, "Method not implemented (entirely missing)" is absolutely dominant, with 239 instances, accounting for 77.85% of its own errors. In contrast, other error types such as "Referencing non-existent classes" (19 instances) and "Referencing non-existent methods or member variables" (15 instances) are very few. This indicates that GPT's main failure mode in the method strategy is to give up on generation when faced with uncertainty.
- **DeepSeek Error Distribution:** Among its 293 errors, "Method not implemented (entirely missing)" is still the primary error, but its number (151 instances) is much lower than GPT's, accounting for 51.54% of its own errors. However, DeepSeek shows a clear tendency for the "Referencing non-existent classes" error, with 73 instances, accounting for 24.91% of its own errors, far higher than GPT's for the same error type.



This differentiated error pattern is highly instructive. In the method strategy, GPT's approach seems to be "better to have nothing than something bad"; when external dependencies are unclear, it is highly likely to choose not to generate any code. In contrast, DeepSeek is more "adventurous." It will more actively attempt to generate code, but due to the lack of sufficient context, it frequently produces hallucinations when inferring external class dependencies, i.e., "inventing" non-existent classes.

Looking back at the class strategy, both models had relatively few errors of "Referencing non-existent classes" (41 for GPT, 20 for DeepSeek) because the complete class definition provided more context. However, under the method strategy, this problem is drastically amplified for DeepSeek, indicating that when processing code snippets with incomplete contextual information, its issue with external dependency hallucinations is more severe than GPT's.

4.8 Data Analysis Summary

Through the data analysis of the method translation strategy and its comparison with the class strategy, we can draw the following conclusions:

- **Translation Granularity Affects Overall Correctness Rate:** Refining the translation task from the class level to the method level can increase the LLM's translation correctness rate from about 61% to about 72%. This proves that optimizing LLM performance by decomposing task complexity is an effective strategy.
- **"Method not implemented" is the Core Hallucination Pattern:** Regardless of the granularity, "Method not implemented (entirely missing)" is the most dominant error type. In the method strategy, which has less contextual information, the models reinforce this "safe failure" strategy, leading to a further increase in the relative proportion of this error. This suggests that enhancing the models' ability to infer and complete code with incomplete information is a key direction for future research.
- **Relative Model Advantages and Weaknesses Vary with the Task:** DeepSeek performs better on complex, high-context class-level translations, while GPT can keep pace or even slightly surpass it on simple, low-context method-level translations. This shows that there is no universally superior model; the choice of model should depend on the specific task scenario and requirements. In low-context scenarios, GPT's main problem is a lack of implementation completeness (tending not to generate), while DeepSeek's main problem is dependency hallucination (tending to generate incorrect dependencies). This finding provides an important basis for the targeted use and improvement of different models. For example, when using DeepSeek for method-level translation, it is necessary to be particularly vigilant and design verification mechanisms to check whether its generated code references non-existent external dependencies.

5 APPROACH

Empirical study has revealed that although splitting the file into methods can solve some of the issues caused bu contextual limitations, the shortcomings of LLMs in understanding the methods of coupling and cross-file calls remain the serious challenges of translating. Another awkward issue is that such an approach also sacrifices a macropic understanding of the whole project, thus cause the more problems of method calling and the fit between method. To alleviate this issue, we attempt to use a series of enhancement methods.

We first improved the prompt of method translating

5.1 Approach A

per

5.2 Approach B

5.3 Approach C

5.4 Discussion

Threats. First,

Limitations. First,

6 CONCLUSIONS

In this paper,